Day 59 - AI Integration and API Planning

AI integration document | API mapping | request/response contracts | reliability and security

Objective and current state

Connect Zecpath AI components across the hiring journey through a proposed service API. The repository currently provides Python modules and a Day 56 simulation runner; it does not contain deployed REST routes.

The simulation calls resume parsing, ATS scoring, screening, HR and technical interview, eligibility, and recommendation components in process. Resume upload is not an HTTP flow, and parsed resume output is not yet mapped to the ATS input schema.

Integration flow and component map

Proposed flow: Hiring UI > authenticated API > workflow coordinator > resume parsing > ATS scoring > screening and interviews > eligibility and recommendation > persistence and audit.

Resume: ResumeEngine.process(file_path) and SmartResumeParser.parse(resume_path) provide parsing entry points. Validate the upload, store it privately, and pass a controlled file reference to the parser.

Scoring and decisions: CandidateMatchingService.generate_score returns an overall percentage, component scores, weights, and explanations. Screening/interview engines manage turns; eligibility and recommendation components return rule outcomes, confidence, risks, reasons, and a human-review flag.

Data storage: SQL schema exists for candidates, jobs, screening sessions, transcripts, and review flags, but the inspected flow does not connect to it. Existing flows use JSON files and JSONL audit output. Add durable interview results, score snapshots, and decisions; store private file references rather than resume contents in records.

Proposed API map and JSON contract

These are design targets, not existing endpoints. Use stable IDs, ISO-8601 timestamps, validated request fields, and one response envelope.

Resume: POST /v1/resumes accepts candidate_id and file; return 202 with job_id. GET /v1/jobs/{id} returns status and parsed result.

ATS: POST /v1/candidate-scores accepts candidate_id, job_id, normalized resume, and job description. Return percentage, skill/experience/education/semantic components, weights, and explanations.

Screening/interviews: POST /v1/screening-sessions and /v1/interview-sessions create sessions; POST each session response with question_id and answer_text, then return the next question or completion state.

Decision: POST /v1/decisions accepts unified_score and eligibility results. Return decision, confidence_score, thresholds, risk_factors, reasons, and human_review_required.

Success envelope:

{"request_id":"req-1","status":"success","data":{...},"error":null}

Processing, reliability, and security plan

Synchronous and asynchronous work

Asynchronous: Resume parsing may exceed a normal request window. Store the upload privately, enqueue a parse job, return 202 Accepted with job_id, and provide a scoped status/result route. Persist job states: queued, processing, completed, or failed.

Synchronous: Run a single ATS score after parsing when latency is bounded. Screening and interview responses should save each turn and return the next question in the same request. Use background jobs for batch scoring and downstream decision preparation; defer decisions until required stages finish.

Errors, retries, and idempotency

Use stable errors for invalid input, unsupported files, parse failures, missing resources, access denial, rate limits, and temporary dependency failures. Client messages should be sanitized; log detail with request_id. Do not retry invalid or unauthorized requests. Retry transient failures a bounded number of times with exponential backoff and jitter, honoring Retry-After.

Require idempotency keys for upload jobs and answer submissions to prevent duplicate sessions or turns. Make workers safe to retry and writes atomic. Expose a safe failure code and retryability in job status.

Security and privacy controls

Authenticate callers with short-lived signed tokens; use service credentials for workers and HTTPS for external traffic. The existing AccessController checks role and candidate scope but expects a trusted Principal, so it does not authenticate callers. Enforce default-deny authorization for every candidate, job, session, transcript, and job-status object.

Validate file type, size, IDs, and payload shape; apply rate limits; check consent before processing; minimize sensitive data in logs; and protect stored resumes and credentials. The current consent manager is in-memory, and the retention component calculates expiry but does not delete records. Persist consent and implement an expiry/deletion worker. Audit access, scoring, overrides, and decisions without logging raw resumes or transcripts.

Implementation order

Define the normalized resume/ATS contract; add authenticated routes and object-level authorization; connect persistence; implement the parse queue and idempotent job states; wire interview turns; then expose decision and human-review actions. Store model versions, score components, and policy thresholds with each result.

Security references: OWASP REST Security Cheat Sheet and OWASP API Security Top 10 - Broken Object Level Authorization.